

UNIVERSITY OF LEEDS

COMP1850 Programming

Week 3, Session 2

Comprehensions and File I/O

UNIVERSITY OF LEEDS

Today's Aims

- Comprehensions - shorthand for building collections with loops
- Reading and cleaning data from files
- Using data structures with read data
- Writing data out to a file from different structures

Comprehension

- The task of iterating over something (a list, a set, a dictionary, a file...) to produce a data structure is very common
- Python provides a specialised construct for this, called a **comprehension**
- Comprehensions are a concise and readable way to create data structures from any iterable thing using a single-line expression

UNIVERSITY OF LEEDS

Comprehension

This combines knowledge of several things we have learned

- Data structures (lists, sets, dictionaries)
- `for` loops
- `if` / Conditional logic

UNIVERSITY OF LEEDS

Comprehension – component parts

expression `for` *item* `in` *iterable-thing* `if` *condition*

- `for` *item* `in` *iterable-thing* retrieves values from anything that supports iteration (e.g., a list, or an opened file)
- `if` *condition* (optional) can be used to filter retrieved values
- *expression* produces a value to use in the new structure
  - *expression* can itself use `if-else` expressions

These three components are enclosed in [ ] to generate a list, { } to generate a set or a dictionary

UNIVERSITY OF LEEDS

How to build a comprehension

1. Start with the loop: `for value in data`
2. Apply the filter(optional): `if value > 0`
3. Save the result: `value * value`
4. Choose the enclosing brackets: `[]`

```
[value * value for value in data if value > 0]
```

UNIVERSITY OF LEEDS

Another Example

```
animals = ['dog', 'cat', 'hamster', 'goldfish']
not_fish = [item for item in animals if item != 'goldfish']
```

You can (hopefully) predict what the new list will contain but you should also test it to be sure

Consider the same code implemented with a `for` loop

UNIVERSITY OF LEEDS

Set comprehension (Dropping duplicates)

{*expression* `for item in` *data-structure* `if` *condition*}

- Uses { } instead of [ ]
- Automatically removes duplicate values and does not guarantee order

```
raw_subjects = ["Math", "science", "MATH", "History", "science"]
clean_subjects = {s.lower() for s in raw_subjects}
# Output: {'math', 'science', 'history'}
```

UNIVERSITY OF LEEDS

Dictionary comprehension (Key-value pairs)

{*key-expr*: *value-expr* for item in *data-struct* if *condition*}

- Uses { } and requires a colon : to separate keys and values
- Useful for mapping or pairing headers to data

```
user_status = {"Alice": "active", "Bob": "inactive", "Charlie":
"active", "David": "expired"}

active_users = {user: status for user, status
    in user_status.items() if status == "active"}

# Output: {'Alice': 'active', 'Charlie': 'active'}
```

UNIVERSITY OF LEEDS

Task 1 - Comprehensions

- Go to `week1.3/session2/task1/`
- Finish implementing all programs in this directory

  1 – list comprehension

  2 – set comprehension

  3 – dictionary comprehension

Files and Paths in Python

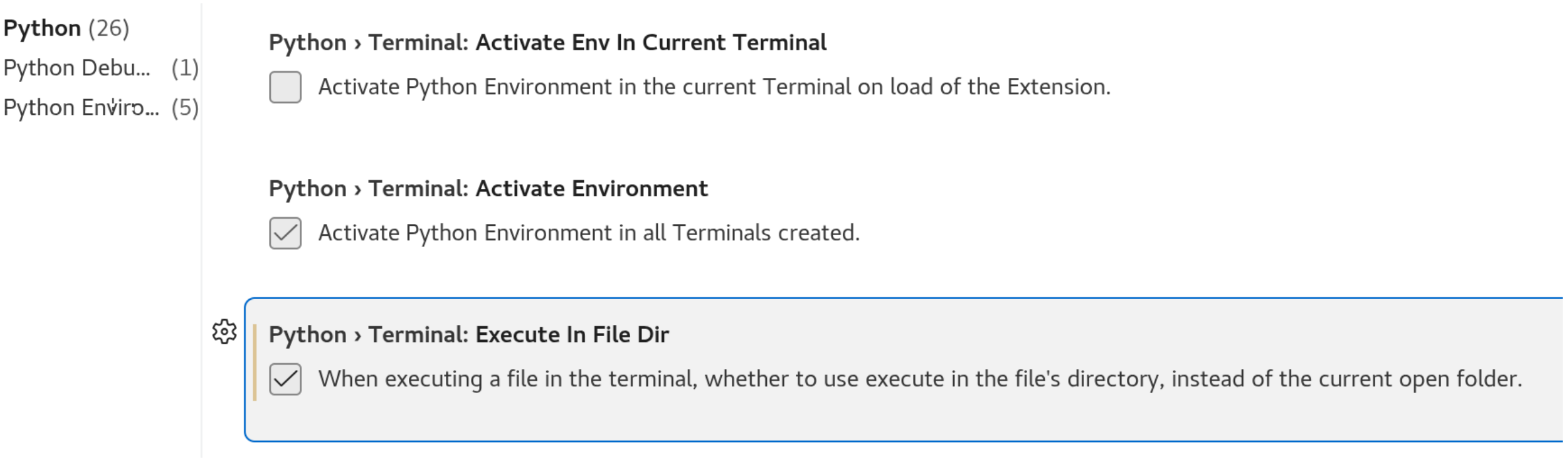
UNIVERSITY OF LEEDS

WARNING: Using the 'Run' button in some IDEs like Codespaces may produce `FileNotFoundError` for external files due to working directory mismatches

- This is because the 'Run' button by default runs programs from a different directory to where the source files are located
- Solution 1: use `cd` in the terminal to navigate to directory containing the program, then run it using `python` command

Solution 2: Codespace Settings

UNIVERSITY OF LEEDS



(Assuming you have the Python extension installed)

UNIVERSITY OF LEEDS

Reading from Files

- Often, you will want to use external data in your programs
- This avoids the need to hard-code data directly into your program, or manually enter it every time the program runs
- Fortunately, file handling is very simple in Python!

UNIVERSITY OF LEEDS

Reading Data from a File: The Four Steps

- Reading data is straightforward and typically involves four steps:
    1. Opening the file
    2. Reading data from the file object
    3. Processing & cleaning the data
    4. Closing the file

- You saw a few examples of this in previous sessions

UNIVERSITY OF LEEDS

Opening a File for Reading

- To access the file contents, a file must be **opened**
- This creates a **file object** which we can read from directly

```
with open("path/to/thefile.txt", "r") as infile:
    # read file contents
    # file closes automatically when we exit block
```

File Objects

- When we open a file, the `open()` function returns a **file object**
- The file object supports a set of **methods** that allow us to read data into our program in different ways
- File objects are **iterable**, so you can use a `for` loop to access data line-by-line – making working with text files straightforward



UNIVERSITY OF LEEDS

Ways to Read Data from a File

There are several ways to read data from a file object:

- `for line in file`
  reads one line at a time, making it memory efficient
- `file.read()`
  reads entire remaining contents as a single string
- `file.readlines()`
  reads entire remaining contents and returns them as a list of strings

Processing and Cleaning the Data

```
with open("student_data.csv", "r") as infile:
    for line in infile:
        print(line)
```

The code above simply *displayed* the data but never stored it within the program itself

Let’s look now at how we can read the data into a structure, typically with some data cleaning

UNIVERSITY OF LEEDS

Cleaning Data

- An example of data cleaning would be to remove any trailing whitespaces and getting it into a useful format
- If we have a list of strings which looks something like:

```
['ID,Name,Mathematics,Science,History\n',
 '1001,Quinn Bell,53,51,88\n'…]
```

We might need to:

- Remove the `\n` from the end
- Split the strings up into individual words

Cleaning Data – List Comprehensions

```
data = ['ID,Name,Math,Science,History\n',
        '1001,Quinn Bell,53,51,88\n',…]
```

As we have a list already, we can use a list comprehension:

```
data = [line.strip().split(",") for line in data]
```

Recall the string methods:

- `strip()` to remove the leading and trailing character \n
- `split(",")` to split a string into individual words using "," separator

UNIVERSITY OF LEEDS

Read in Data to Data Structures

- For simplicity we are focusing on reading in data to lists because these are very flexible and can be used for basically any data
- However, you will sometimes find it more useful to use a dictionary or another more specialised data structure
- Once you have the data in your program in a suitable data structure, you can use any of the things you've already learned to process the data

Task 2: Reading from a File

UNIVERSITY OF LEEDS

- Go to `week1.3/session2/task2/`
- Pick one or two of these to work on in class:

  1 – word_counter

  2 – log_data

  3 – sales_data

- Do the remaining ones in private study time, for more practice

UNIVERSITY OF LEEDS

Writing Data to a File

Just like reading data in, writing data out is very useful.

It allows you to:

- Save the program state
- Create formatted files for other purposes
- Save processed data for review

Writing Data Out - how

UNIVERSITY OF LEEDS

The process is very similar to reading in:

- Open the file
- Ensure data is in the right format
- Write the data
- Close the file

Same steps, slightly different order!

Writing Data Out - code

```
with open("path/to/thefile.txt", "w") as outfile:
...
```

What is different to last time?

UNIVERSITY OF LEEDS

Different modes with open()

There are different **modes** when opening a file with the `open()` function

The most common ones are:

- `"r"` – `read`
- `"w"` – `write` (overwrite the current contents)
- `"a"` – `append` (add to the current contents)

When we open a file, it's important to pick the correct **mode**

This also decides the behaviour if the file doesn't exist:

- in `read` mode, it triggers an error
- in `write` and `append` modes, it creates the file

UNIVERSITY OF LEEDS

Formatting the Data for Writing to a File

When we write the data out to a file, we need to consider

- what are we using the data for
- and how should we format it.

Does it need to be human readable?

Is it going to be loaded into another program?

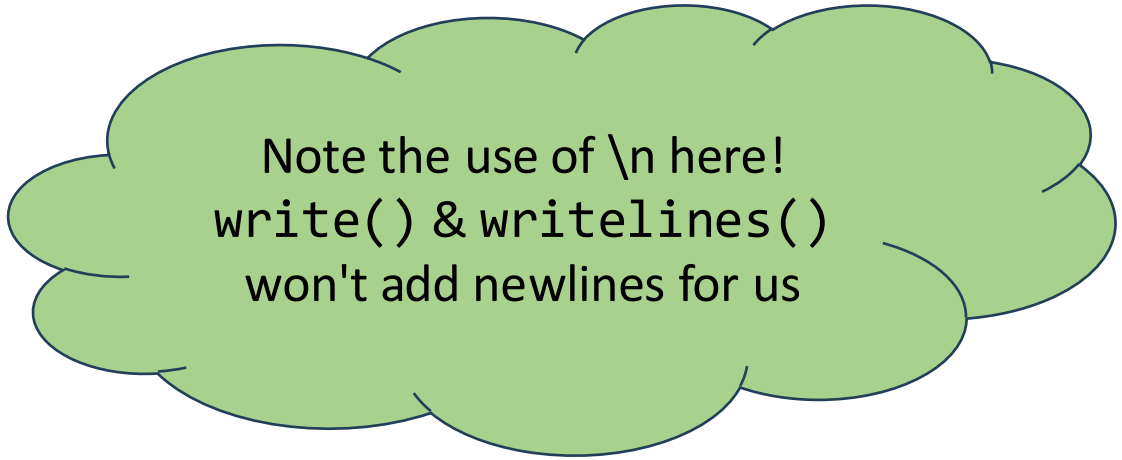
Is there a specific format it needs to follow?

UNIVERSITY OF LEEDS

Writing the Data to a File

Once you know what format you need to write, it's simple:

```
with open("path/to/thefile.txt", "w") as outfile:
      outfile.write("Some literal text\n")
      outfile.write(string_variable + "\n")
      outfile.writelines(["this\n", "also\n", "works\n"])
```



UNIVERSITY OF LEEDS

Task 3: Writing to a File

- Go to `week1.3/session2/task3/`
- Pick one of the programs in the directory to implement in class, then do the other in your own time

  1 – user_info

  2 – average_grade

UNIVERSITY OF LEEDS

Bringing Skills Together

You've seen a load of different skills, so it's time to get some practice bringing them all together:

- `try…except…` for robust code
- Python's collections and comprehensions
- Loops
- File I/O

This is also a good chance for you to make sure you understand the different topics and ask any questions you have!

UNIVERSITY OF LEEDS

Task 4: Reading and Writing PGM images

- Go to `week1.3/session2/task4/`
- Finish implementing all programs in the directory

  1 – Displaying a PGM image information

  2 – Validate a PGM image

  3 – Invert a PGM image

UNIVERSITY OF LEEDS

Summary

We have

- Introduced comprehensions in Python
- Seen examples of list, set, and dictionary comprehensions
- Introduced file reading and writing
- Seen an example of cleaning data from a file